

Attachment Processing Implementation Plan

> Extending the autopilot to process PDFs and images sent via Telegram, extract their content, and persist to transcript for cross-session memory.

1. Why

Problem Statement

Currently, when a governor sends a file attachment (PDF, image) via Telegram to the autopilot:

1. **PDFs** — cannot be read at all. The autopilot has no PDF text extraction capability.
2. **Images** — QR codes are scanned via pyzbar, and Grok vision provides scene descriptions, but general OCR (reading text in images) is not performed.
3. **Persistence** — even when content is extracted within a session, it is not saved to the transcript repo in a structured way that enables cross-session recall. If the governor asks "remember that PDF I sent last week?", the autopilot has no way to retrieve it.
4. **Voice conversations** — the governor wants to send a file, have it processed, and then discuss it via voice. Without extraction + persistence, the voice conversation has nothing to reference.

Vision

A governor can send any file attachment via Telegram and the autopilot will:

1. **PDFs** — extract all text content using a PDF parser
2. **Images** — run OCR to extract visible text AND Grok vision to describe the scene
3. **Persist** — write all extracted content to the session transcript in ``truesight_autopilot_transcript``
4. **Recall** — in a future session, the governor can say "remember that PDF I sent?" and the autopilot reads it back from the transcript

This closes the loop between file-based communication and conversational AI, making Telegram a true multimodal interface to the DAO.

Success Criteria

- Governor sends a PDF → autopilot extracts text → saves to transcript → can discuss in same session
- Governor sends an image → autopilot runs OCR + Grok vision → saves to transcript → can discuss
- Governor starts a new session and says "remember that PDF from last week?" → autopilot reads transcript and recalls the content
- All extracted content is stored in the existing transcript format (no new storage systems)
- No new API keys required beyond what's already configured (Grok vision already works)

2. Implementation Details

Architecture

The implementation adds three new capabilities to the autopilot, all accessible as helper scripts that the Telegram message handler can invoke.

Components

1. PDF Text Extraction — scripts/extract_pdf_text.py

- **Library:** pymupdf (a.k.a. fitz) — pure Python, no system deps, fast, handles most PDFs
- **Fallback:** pdminer.six if pymupdf fails on a particular PDF
- **Output:** Plain text extracted from all pages, with page markers
- **Edge cases:** Scanned PDFs (image-only) → detected and flagged for OCR path

2. Image OCR — scripts/ocr_image.py

- **Library:** pytesseract (Python wrapper for Tesseract OCR engine)
- **System dep:** tesseract-ocr package (apt-get installable)
- **Preprocessing:** Pillow-based image enhancement (grayscale, threshold, contrast) before OCR
- **Output:** Extracted text with bounding box coordinates
- **Edge cases:** Low-light, blurry, skewed images → flag quality issues

3. Transcript Persistence — scripts/append_to_transcript.py

- Reads the current session transcript file from truesight_autopilot_transcript
- Appends a structured section with the extracted content
- Commits and pushes to the transcript repo

4. Telegram Handler Integration

The existing Telegram message handler (app/telegram_bot.py) already receives file attachments. The integration point is:

1. Detect file type from MIME or extension
2. Download to /tmp/autopilot_uploads/ (already exists)
3. Route to appropriate extractor (PDF or image)
4. For images: run OCR + Grok vision (Grok is already wired)
5. Combine results
6. Call transcript persistence
7. Return summary to the governor in the chat

Key Decisions

Decision	Choice	Rationale
PDF library	pymupdf (fitz)	Pure Python, fast, no system deps, handles most PDFs
OCR engine	Tesseract via pytesseract	Industry standard, free, well-supported
Vision model	Grok (already configured)	Already wired in the autopilot, no new API key needed
Storage	Transcript repo (existing)	No new storage system; leverages existing cross-session persistence
Processing location	Autopilot EC2	Files are already downloaded there; no network transfer needed

Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
pymupdf fails on complex PDF	Low	Medium	Fallback to pdfminer.six
Tesseract not installed on EC2	Medium	High	Add to deploy.sh; document as dependency
OCR quality poor on dark/blurry photos	Medium	Low	Flag quality in output; Grok vision as backup
Large PDFs (>50 pages) timeout	Low	Medium	Page limit; process first N pages
Transcript repo commit conflicts	Low	Low	Append-only; use timestamped sections

Dependencies

Dependency	Type	Status
pymupdf	Python package	Needs install
pytesseract	Python package	Needs install
tesseract-ocr	System package	Needs install
pdfminer.six	Python package (fallback)	Needs install
Grok API key	Credential	Already configured
Transcript repo write access	GitHub PAT	Already configured
Telegram bot token	Credential	Already configured

3. Execution Roadmap

Phase 1: Foundation — Scripts & Dependencies

- [] Install system deps on autopilot EC2: `apt-get install -y tesseract-ocr`
- [] Install Python deps: `pip install pymupdf pytesseract pdfminer.six`
- [] Create `scripts/extract_pdf_text.py` — PDF text extraction with `pymupdf`, fallback to `pdfminer`
- [] Create `scripts/ocr_image.py` — Tesseract-based OCR with Pillow preprocessing
- [] Create `scripts/append_to_transcript.py` — Append structured content to session transcript
- [] **Checkpoint:** All three scripts work standalone via CLI

Phase 2: Integration — Telegram Handler

- [] Modify Telegram message handler to detect file attachments (PDF vs image)
- [] Wire PDF path → `extract_pdf_text.py` → results
- [] Wire image path → `ocr_image.py` + Grok vision → combined results
- [] Wire combined results → `append_to_transcript.py`
- [] Return summary to governor in chat ("Extracted 3 pages from PDF, saved to transcript")
- [] **Checkpoint:** End-to-end flow works: send file → processed → saved → discussed

Phase 3: Cross-Session Recall

- [] Add a `recall_attachment` intent in the autopilot's system prompt
- [] When governor says "remember that PDF from last week", search transcript repo for matching session
- [] Read and return the extracted content
- [] **Checkpoint:** Governor can reference attachments across sessions

Phase 4: Polish & Edge Cases

- [] Handle scanned PDFs (image-only) → route to OCR path
- [] Handle password-protected PDFs → return error message
- [] Handle corrupt/invalid files → graceful error
- [] Add page/quality limits to prevent runaway processing
- [] Add progress feedback during long extractions
- [] **Checkpoint:** All edge cases handled gracefully

Last updated: 2026-06-07